

FORMATION EMBARQUÉE

TD 03 Arduino

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 03 — Arduino : énoncés et corrigés détaillés

Tous les corrigés sont testables sur <https://wokwi.com> sans matériel. Aucun corrigé n'utilise `delay()` dans la boucle : c'est le critère n°1.

Exercice 1 — Chenillard 4 LED, vitesse au potentiomètre, sans

`delay()`

Corrigé

```
const uint8_t LEDS[] = {8, 9, 10, 11};
const uint8_t NB_LEDS = sizeof LEDS / sizeof LEDS[0];
const uint8_t POT = A0;

uint8_t index_led = 0;
uint32_t derniere_avance = 0;

void setup() {
    for (uint8_t i = 0; i < NB_LEDS; i++) pinMode(LEDS[i], OUTPUT);
}

void loop() {
    // Période recalculée à CHAQUE tour : la vitesse réagit immédiatement.
    // map() : 0..1023 → 50..500 ms
    uint16_t periode = map(analogRead(POT), 0, 1023, 50, 500);

    uint32_t maintenant = millis();
    if (maintenant - derniere_avance >= periode) {
        derniere_avance = maintenant;

        digitalWrite(LEDS[index_led], LOW);           // éteindre l'actuelle
        index_led = (index_led + 1) % NB_LEDS;         // avancer (retour à 0 après 3)
        digitalWrite(LEDS[index_led], HIGH);           // allumer la suivante
    }
}
```

Points de correction : - Le tableau `LEDS[]` + `NB_LEDS` calculé par `sizeof` : ajouter une 5e LED = une seule ligne à changer. Quatre `digitalWrite` copiés-collés = zéro pointé en style. - `uint32_t` pour tout ce qui touche `millis()` (piège du `int` 16 bits). - La soustraction `maintenant - derniere_avance` survit au débordement de `millis()` (au bout de ~49 jours) — un `if (millis() > prochain_top)` non.

Exercice 2 — Compteur de passages (ultrason + OLED + bouton

Corrigé (structure complète)

```

#include <Wire.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 oled(128, 64, &Wire, -1);

const uint8_t TRIG = 3, ECHO = 4, BTN_RAZ = 2;
const uint16_t SEUIL_CM = 50;           // "passage" = obstacle à moins de 50 cm

uint16_t compteur = 0;
bool      objet_present = false;        // pour ne compter qu'UNE fois par passage
uint32_t derniere_mesure = 0, derniere_maj_oled = 0;

// -- anti-rebond bouton (scrutation, pas d'interruption nécessaire ici)
bool      btn_stable = false, btn_brut_prec = false;
uint32_t btn_t_chgt = 0;

float mesurer_cm() {
    digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
    uint32_t duree = pulseIn(ECHO, HIGH, 30000UL); // timeout 30 ms : JAMAIS sans
    return (duree == 0) ? 999.0f : duree / 58.0f;   // 0 = timeout → "loin"
}

bool bouton_appuye(uint32_t maintenant) {
    bool brut = (digitalRead(BTN_RAZ) == LOW);
    if (brut != btn_brut_prec) { btn_t_chgt = maintenant; btn_brut_prec = brut; }
    if (maintenant - btn_t_chgt > 20 && brut != btn_stable) {
        btn_stable = brut;
        return btn_stable;           // true seulement sur le FRONT d'appui
    }
    return false;
}

void setup() {
    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);
    pinMode(BTN_RAZ, INPUT_PULLUP);
    oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(3);
}

void loop() {
    uint32_t maintenant = millis();

    // 1) Mesure toutes les 60 ms (l'HC-SR04 n'aime pas être mitraillé)
    if (maintenant - derniere_mesure >= 60) {
        derniere_mesure = maintenant;
        bool proche = (mesurer_cm() < SEUIL_CM);
        if (proche && !objet_present) compteur++; // front d'ENTRÉE seulement
        objet_present = proche;
    }
}

```

```

}

// 2) Bouton de remise à zéro
if (bouton_appuye(maintenant)) compteur = 0;

// 3) OLED rafraîchi 5 fois/s (le rafraîchir à chaque tour ralentit tout)
if (maintenant - derniere_maj_oled >= 200) {
    derniere_maj_oled = maintenant;
    oled.clearDisplay();
    oled.setCursor(10, 20);
    oled.print(compteur);
    oled.display();
}
}

```

Points de correction : - **Détection de front** (`proche && !objet_present`) : sans elle, un objet immobile devant le capteur incrémente en continu. - **Timeout sur** `pulseIn` : sans lui, pas d'écho = boucle figée 1 s. - Trois activités (mesure, bouton, affichage) à trois cadences différentes dans une seule boucle — c'est exactement l'architecture attendue.

Exercice 3 — Thermostat à hystérésis en machine d'états

Corrigé

```
#include <DHT.h>

DHT dht(2, DHT22);
const uint8_t RELAIS = 7, POT_CONSIGNE = A0;
const float HYSTERESIS = 0.5f;

enum class Etat : uint8_t { Repos, Chauffe, Default };
Etat etat = Etat::Repos;

float consigne = 20.0f, temperature = NAN;
uint8_t echecs_lecture = 0;
uint32_t derniere_lecture = 0;

void setup() {
    pinMode(RELAIS, OUTPUT);
    digitalWrite(RELAIS, LOW);          // état SÛR au démarrage
    dht.begin();
    Serial.begin(115200);
}

void loop() {
    uint32_t maintenant = millis();

    // Consigne : 10..30 °C selon le potentiomètre
    consigne = 10.0f + analogRead(POT_CONSIGNE) * (20.0f / 1023.0f);

    // Le DHT22 se lit au plus toutes les 2 s
    if (maintenant - derniere_lecture >= 2000) {
        derniere_lecture = maintenant;
        float t = dht.readTemperature();
        if (isnan(t)) {
            if (++echecs_lecture >= 3) etat = Etat::Default;    // 3 échecs = panne
        } else {
            echecs_lecture = 0;
            temperature = t;
        }
    }

    switch (etat) {
    case Etat::Repos:
        digitalWrite(RELAIS, LOW);
        // On enclenche SOUS consigne - hystérésis (pas à la consigne pile)
        if (!isnan(temperature) && temperature < consigne - HYSTERESIS)
            etat = Etat::Chauffe;
        break;

    case Etat::Chauffe:
        digitalWrite(RELAIS, HIGH);
        // On coupe AU-DESSUS de consigne + hystérésis
        if (isnan(temperature) || temperature > consigne + HYSTERESIS)
```

```

        etat = Etat::Repos;
    break;

    case Etat::Default:
        digitalWrite(RELAIS, LOW);          // panne capteur → chauffage COUPÉ
        // Sortie de défaut uniquement si le capteur répond à nouveau
        if (echecs_lecture == 0 && !isnan(temperature)) etat = Etat::Repos;
        break;
    }
}

```

Pourquoi l'hystérésis : sans elle, autour de la consigne le relais « bat » (clignote) à chaque dixième de degré — usure mécanique et bruit. La bande morte de $\pm 0,5$ °C impose un cycle lent : c'est le comportement de tous les thermostats réels (et on le retrouvera à l'identique en SCL au TD 07).

Le point le plus noté : l'état `Default`. Un thermostat qui garde le chauffage allumé quand son capteur meurt est un danger. Réflexe : *que fait mon système quand chaque entrée tombe en panne ?* — question à se poser dans CHAQUE projet, jusqu'aux automates (modules 07-08).

Exercice 4 (complément) — Lecture de code : trouver les 5 défauts

Énoncé. Ce sketch « marche presque ». Liste ses défauts.

```

int compteur = 0;

void setup() {
    attachInterrupt(digitalPinToInterrupt(2), tic, FALLING);
    Serial.begin(9600);
}

void tic() {
    compteur++;
    Serial.println("tic!");
}

void loop() {
    delay(1000);
    Serial.println(compteur);
    if (compteur == 60) compteur = 0;
}

```

Corrigé

1. `Serial.println` dans l'ISR : les fonctions série utilisent des interruptions et des tampons — appelées depuis une ISR, elles peuvent bloquer définitivement. Une ISR pose un drapeau, point.
2. `compteur` non `volatile` : partagé ISR/boucle, le compilateur peut en garder une copie en registre dans `loop()`.
3. **Lecture non atomique** : `compteur` est un `int` 16 bits sur Uno — sa lecture dans `loop()` peut être coupée par l'ISR. Il faut `noInterrupts()/interrupts()` autour de la lecture (et de la remise à 0).

4. `if (compteur == 60)` : l'ISR peut faire passer le compteur de 59 à 61 entre deux tests (deux fronts rapides) → la condition n'est jamais vraie. Écrire `>= 60`.
 5. `pinMode(2, INPUT_PULLUP)` **absent** : broche flottante → fronts parasites en rafale. (Bonus : aucun anti-rebond si la source est un bouton mécanique.)
-

Auto-évaluation avant le TP 1

Sans notes : écrire le motif `millis()` de tête ; expliquer front vs niveau ; justifier l'hystérésis ; citer les règles d'une ISR ; expliquer pourquoi trois cadences différentes peuvent cohabiter dans une seule `loop()`.

➡ Passe au [TP 1 — Station météo](#).